Le développement logiciel à l'ère de l'IA

Cours 1 — Les assistants IA : bases des LLM et du *prompting*

Pierre Sutra

Télécom SudParis

CIA 4101 (Certificat IA) - Partie Informatique

version 1.0

Le développement logiciel à l'ère de l'IA

Comprendre et utiliser les LLMs pour le développement logiciel, du *prompt* aux systèmes multi-agents.

IA symbolique et LLMs

Fiabiliser le raisonnement des LLMs en le faisant valider par des solveurs logiques : l'IA neurosymbolique. *B. Raddaoui*

Le développement logiciel à l'ère de l'IA

Comprendre et utiliser les LLMs pour le développement logiciel, du *prompt* aux systèmes multi-agents.

IA symbolique et LLMs

Fiabiliser le raisonnement des LLMs en le faisant valider par des solveurs logiques : l'IA neurosymbolique. *B. Raddaoui*

Les objectifs de ce cours :

- comprendre ce qu'est un LLM : un générateur probabiliste, sans état, avec ses limites
- rédiger des *prompts* efficaces (*few-shot*, *chain-of-thought*, *self-consistency*, *reflexion*), et les évaluer
- donner des capacités au modèle : appel d'outils, ancrage par la recherche (RAG), protocole MCP
- comprendre et écrire un agent : la boucle, le *harness*, la gestion du contexte
- collaborer avec un agent de code : niveaux d'autonomie, plan, tests, revue du *diff*
- maîtriser les risques : sécurité, coût, responsabilité
- diriger plusieurs agents, et anticiper l'évolution du métier de développeur

Organisation

- 4 cours + 1 intervention extérieure
  - Masoud Saeida Ardekani, Staff Software Engineer, Google (Mountain View, CA)
- des transparents
- des exercices en Python à faire chez vous (utilisables avec un LLM local)
  - faites-les : c'est *très* formateur
  - deux niveaux de compréhension : m'écouter, et coder ces choses vous-mêmes
- des quiz durant le cours

Fil conducteur

Cours 1 (aujourd'hui)	les *assistants* IA à vos côtés : bases du *prompting* et du *prompt engineering* ; l'humain boucle chaque itération
Cours 2	appel d'outils et ancrage (*tool calling*, *grounding*) : donner des *capacités* au modèle (appel de fonctions, RAG, écosystème)
Cours 3	développement agentique : la boucle se referme d'elle-même
Cours 4	plusieurs agents, et l'avenir du développement logiciel

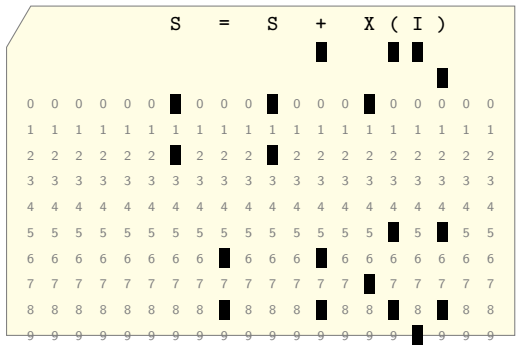
Plan

1. Le nouveau paradigme de développement
2. Les bases des LLM (pour le développeur)
3. Les bases du *prompting*
4. L'essentiel du *prompt engineering*
5. Pratiques du développement assisté par l'IA

« Quelle part de votre code avez-vous *écrite* aujourd'hui — et quelle part avez-vous *acceptée* ? »

- `Tab` → une ligne entière ; une phrase en français → une fonction
- à partir de quand écrire du code devient-il accepter du code ?

1. cartes perforées (trous physiques)



la somme d'une liste

1. cartes perforées (trous physiques)
2. assembleur (mnémoniques)

```
        xor  eax, eax
        mov  rcx, rsi
boucle: add  eax, [rdi]
        add  rdi, 4
        loop boucle
```

la somme d'une liste

1. cartes perforées (trous physiques)
2. assembleur (mnémoniques)
3. langages de haut niveau (proches du problème)

```
int somme(int *x, int n) {
  int s = 0;
  for (int i = 0; i < n; i++)
    s += x[i];
  return s;
}
```

la somme d'une liste

1. cartes perforées (trous physiques)
2. assembleur (mnémoniques)
3. langages de haut niveau (proches du problème)
4. objets et *frameworks* (abstractions réutilisables)

```
class Liste {
  private List<Integer> xs;
  int somme() {
    return xs.stream()
      .mapToInt(Integer::intValue)
      .sum();
  }
}
```

la somme d'une liste

1. cartes perforées (trous physiques)
2. assembleur (mnémoniques)
3. langages de haut niveau (proches du problème)
4. objets et *frameworks* (abstractions réutilisables)
5. écosystèmes ouverts (paquets, services)

```
$ pip install numpy
```

```
import numpy as np
s = np.sum(x)
```

la somme d'une liste

1. cartes perforées (trous physiques)
2. assembleur (mnémoniques)
3. langages de haut niveau (proches du problème)
4. objets et *frameworks* (abstractions réutilisables)
5. écosystèmes ouverts (paquets, services)
6. **développement assisté par l'IA** : la nouvelle vague

« Écris une fonction Python qui renvoie la somme d'une liste d'entiers. »

LLM

```
def somme(x: list[int]) -> int:
    return sum(x)
```

la somme d'une liste

À chaque changement de paradigme :

nouvelle couche d'abstraction → les développeurs résistent (« ce n'est pas de la vraie programmation ») → la productivité bondit → la notion de « compétence » se redéfinit

- de l'assembleur à Fortran : les programmeurs doutaient qu'un compilateur produise un code aussi efficace que le leur, écrit à la main [1]
- puis C contre l'assembleur, Java contre C, « le front-end, ce n'est pas de l'ingénierie »

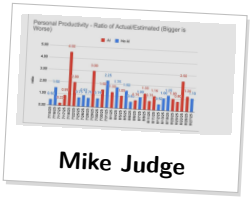
⇒ **le développement assisté par l'IA est une nouvelle vague, pas la dernière**

1. J. Backus. « The History of FORTRAN I, II, and III ». *ACM SIGPLAN Notices* (1978).

- 2017–2020 → fondations : le Transformer, des modèles de langage entraînés aussi sur du code (GPT-2, GPT-3)
- 2021–2022 → auto-complétion générative dans l'éditeur (Codex, GitHub Copilot), programmation compétitive (AlphaCode)
- 2023 → assistants conversationnels, qui cherchent dans le dépôt les extraits utiles (ChatGPT, Copilot Chat, Cursor)
- 2024 → premiers agents, interfaces générées, modèles de raisonnement (Devin, Replit Agent, Bolt.new, o1)

- début 2025 → agents dans le terminal et l'éditeur, sur le poste du développeur (Claude Code, Codex CLI, mode agent de Cursor), modèles de raisonnement ouverts (DeepSeek-R1)
- mi-2025 → agents asynchrones dans le cloud, qui rendent une *pull request* (Jules, Codex, agent de codage de Copilot), routage entre modèle rapide et modèle de raisonnement (GPT-5)
- 2025–2026 → orchestrateurs de plusieurs agents en parallèle, un par branche (Conductor)

Sceptiques



Mike Judge

The Future of Everything is Lies, I Guess

Aphyr

Apr 2026

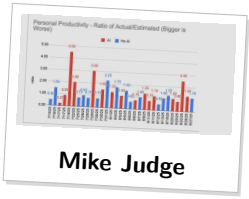
Agentic Coding is a Trap

Remaining vigilant about cognitive debt and atrophy.

Lars Faye

Les avis sont partagés

Sceptiques



Mike Judge

The Future of Everything is Lies, I Guess

Aphyr

Apr 2026

Agentic Coding is a Trap

Remaining vigilant about cognitive debt and atrophy.

Lars Faye

Optimistes

With AI You Need to Think Much Bigger!

Posted on 10 March 2025 by admin

Boz

My AI Skeptic Friends Are All Nuts

Thomas Ptacek

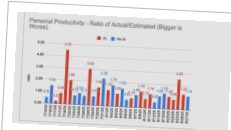
AI demands more engineering discipline. Not less

Charity Majors

Les avis sont partagés

1. Le nouveau paradigme de développement

Sceptiques



Mike Judge

What's Easy Now? What's Hard Now?

Marc Brooker

The Future of Everything is Lies, I Guess

Aphyr

OPINION
GUEST ESSAY

You're a Computer Science Major. Don't Panic.

Nov. 12, 2025

M. Shaw & M. Hilton

Apr 2026

Agentic Coding is a Trap

Remaining vigilant about cognitive debt and atrophy.

Lars Faye

Mi-figue, mi-raisin



Luc Bonnin

Understand and explore the bleeding edge, b
adopt a dampened one

Ozzie

Engineering management after the cost of code collapsed

Karim Jedda

Optimistes

With AI You Need to Think Much Bigger!

Posted on 10 March 2025 by admin

Boz

Using AI to write better code more slowly

Nolan

My AI Skeptic Friends Are All Nuts

Thomas Ptacek

AI demands more engineering discipline. Not less

Charity Majors

Ce que les LLM savent faire . . . et ne savent pas faire

Ils savent

- code répétitif, squelettes de projet
- traduire d'un langage à un autre
- expliquer du code en langage courant
- écrire tests et documentation
- suggérer des refactorisations
- compléter une implémentation partielle

$\Rightarrow$ tâches *génératives* et *transformatives*

Ce que les LLM savent faire . . . et ne savent pas faire

Ils savent

- code répétitif, squelettes de projet
- traduire d'un langage à un autre
- expliquer du code en langage courant
- écrire tests et documentation
- suggérer des refactorisations
- compléter une implémentation partielle

$\Rightarrow$ tâches *génératives* et *transformatives*

Ils ne savent pas

- garantir la correction (API hallucinées, erreurs de logique subtiles)
- garder le contexte sur de longues distances
- deviner l'intention métier
- faire des arbitrages d'architecture
- remplacer la relecture et le jugement

connaître les limites compte autant que connaître les capacités

écrire chaque ligne	$\rightarrow$	définir l'intention
mémoriser des API	$\rightarrow$	décrire son besoin
déboguer la syntaxe	$\rightarrow$	déboguer les *prompts* et les spécifications
valider le résultat	$\rightarrow$	**valider le résultat**

ce qui compte : **le goût, le jugement, la pensée informatique**

écrire chaque ligne	$\rightarrow$	définir l'intention
mémoriser des API	$\rightarrow$	décrire son besoin
déboguer la syntaxe	$\rightarrow$	déboguer les *prompts* et les spécifications
valider le résultat	$\rightarrow$	**valider le résultat**

ce qui compte : **le goût, le jugement, la pensée informatique**

- le goût : reconnaître une bonne solution (simple, lisible, idiomatique, bien découpée) et refuser celle qui marche mais est mal conçue

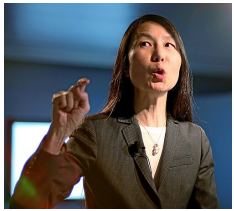
écrire chaque ligne	$\rightarrow$	définir l'intention
mémoriser des API	$\rightarrow$	décrire son besoin
déboguer la syntaxe	$\rightarrow$	déboguer les *prompts* et les spécifications
valider le résultat	$\rightarrow$	**valider le résultat**

ce qui compte : **le goût, le jugement, la pensée informatique**

- le goût : reconnaître une bonne solution (simple, lisible, idiomatique, bien découpée) et refuser celle qui marche mais est mal conçue
- le jugement : savoir quand faire confiance au modèle, quoi vérifier, et quelles décisions ne pas lui déléguer (sécurité, architecture, compromis)

- la compétence fondamentale n'est pas de coder, c'est la *computational thinking* : **décomposition, abstraction, reconnaissance de motifs, conception d'algorithmes**
- les changements de paradigme ont changé les *outils*, pas la *pensée*
- à mesure que l'IA absorbe les couches mécaniques, la pensée est ce qui vous reste
- la décomposition de Wing à l'ère de l'IA : l'intention, des morceaux à la taille d'un *prompt*, la vérification

« Computational thinking is a fundamental skill for everyone, not just for computer scientists. » [1]



Jeannette M. Wing, ex-*Corporate VP, Head of Research* chez Microsoft

photo : World Economic Forum, CC BY-SA 2.0

1. J. M. Wing. « Computational Thinking ». *Communications of the ACM* (2006).

À retenir

- l'IA est la dernière abstraction en date qui augmente la productivité ; chaque vague a été accueillie avec scepticisme
- connaître les capacités des LLM *et* leurs limites
- le rôle du développeur se déplace vers l'intention, la validation et la conception de systèmes

1. Le nouveau paradigme de développement
2. Les bases des LLM (pour le développeur)
3. Les bases du *prompting*
4. L'essentiel du *prompt engineering*
5. Pratiques du développement assisté par l'IA

Modèles de langage : l'idée centrale

LLM = *Large Langage Model*, FM = *foundation model*

- un modèle de langage attribue une probabilité à une séquence de *tokens*
- LLM : la prédiction du *token* suivant, à grande échelle
- génération *autorégressive* : tirer x_{t+1} selon $P(x_{t+1} \mid x_1, \ldots, x_t)$, l'ajouter à la séquence, recommencer

- elle règle le caractère aléatoire du tirage
  - basse : déterministe, répétitif
  - haute : varié, « créatif »
- *top-p* et *top-k* : d'autres façons de tronquer la distribution avant le tirage [1]
- pourquoi la température intéresse les développeurs :
  - une sortie quasi déterministe pour générer du code
  - de la variété pour les tâches de rédaction

1. *top-k* : on ne garde que les *k tokens* les plus probables ; *top-p* (*nucleus sampling*) : on garde les *tokens* les plus probables jusqu'à ce que leur probabilité cumulée atteigne *p*. Dans les deux cas, on renormalise avant de tirer.

- le texte est découpé en *tokens* avant que le modèle ne le voie
- **BPE** (*Byte Pair Encoding*) [1] : un compromis entre un modèle au niveau des caractères et un modèle au niveau des mots
  - à l'origine, de la compression : remplacer la paire d'octets la plus fréquente par un seul octet inutilisé, et recommencer
  - « segmenter les mots rares en sous-mots adaptés suffit au réseau de traduction neuronale »
- conséquences pour les développeurs :
  - le contexte se compte en *tokens*, pas en mots ($\approx$ 0,75 mot par *token* en anglais)
  - les particularités du *tokenizer* (espaces, Unicode, nombres, code) jouent sur le coût et le comportement

1. A. Radford et al. *Language Models are Unsupervised Multitask Learners*. OpenAI, 2019 ; R. Sennrich et al. « Neural Machine Translation of Rare Words with Subword Units ». *ACL*. 2016.

- tout ce que le modèle voit en même temps : *prompt* système + conversation + documents récupérés + texte généré (on clarifie ce point plus loin)
- elle est **finie** : au-delà, l'ancien contexte est supprimé ou résumé — « le modèle oublie »
  - le modèle ne garde rien d'un appel à l'autre : on lui renvoie à chaque fois tout l'historique, qui doit tenir dans la fenêtre (cours 2)
- conséquences pratiques :
  - un compromis entre la taille du *prompt* et la qualité
  - la raison d'être du RAG (cours 2)
  - pourquoi les agents doivent résumer leur historique (cours 3)

- **entraîner** un modèle = ajuster ses paramètres (les poids du réseau, des milliards) pour qu'il prédise au mieux le *token* suivant sur un immense corpus
  - très coûteux, fait une fois (puis un post-entraînement, voir l'alignement)
- données d'entraînement : Common Crawl comme point de départ, plus des corpus de qualité (WebText [1] = tous les liens sortants de Reddit avec au moins 3 de karma, des livres, Wikipédia en anglais)
- entraînés sur des millions de pages web, les LLM résolvent questions-réponses, traduction, résumé, mieux que les systèmes supervisés de l'état de l'art (auparavant : Wikipédia, romans, ou une seule tâche)

Vu la taille de ces modèles, les entraîner demande des systèmes répartis (le parameter server [2], ou plus récemment Ray [3]).

1. A. Radford et al. *Language Models are Unsupervised Multitask Learners*. OpenAI, 2019.
2. M. Li et al. « Scaling Distributed Machine Learning with the Parameter Server ». *OSDI*. 2014.
3. P. Moritz et al. « Ray : A Distributed Framework for Emerging AI Applications ». *OSDI*. 2018.

L'inférence

- **inférence** = utiliser le modèle entraîné : pour une entrée (le *prompt*), calculer la distribution du *token* suivant
- une passe avant uniquement : les poids restent figés, rien n'est appris
  - même le *few-shot* (section suivante) se passe à l'inférence : les exemples sont dans le *prompt*, pas dans les poids

- **inférence** = utiliser le modèle entraîné : pour une entrée (le *prompt*), calculer la distribution du *token* suivant
- une passe avant uniquement : les poids restent figés, rien n'est appris
  - même le *few-shot* (section suivante) se passe à l'inférence : les exemples sont dans le *prompt*, pas dans les poids
- répétée pour chaque *token* généré et pour chaque utilisateur : à très grande échelle, facturée au *token*
- la génération s'arrête quand :
  - le modèle produit un *token* spécial de fin (*end of sequence*), appris à l'entraînement : il « décide » que sa réponse est finie
  - ou la limite de *tokens* fixée par l'appelant est atteinte (`max_tokens`) : la réponse est alors tronquée
  - ou une séquence d'arrêt choisie par l'appelant apparaît (*stop sequence*)
- c'est ce qui se passe chaque fois que vous utilisez un chatbot

- la prédiction du *token* suivant optimise le *token probable*, pas le *token vrai*
- plausible mais faux : API, code, faits, citations
- les détecter : vérifier, demander les sources, recouper
- les atténuer : *few-shot*, *self-consistency* (section suivante), génération ancrée (RAG, cours 2)

- un modèle de base *complète* du texte ; suivre des instructions demande un post-entraînement
  - *instruction tuning* : entraîner sur des paires (instruction, réponse attendue)
  - RLHF (*reinforcement learning from human feedback*) : renforcer les réponses que des humains préfèrent
- aligner = orienter le comportement vers l'utile, l'honnête, le sûr
- conséquences : les refus (« je ne peux pas faire cela »), et les *jailbreaks* pour les contourner
- c'est pourquoi un même modèle sous-jacent se comporte différemment d'un chatbot à l'autre

À retenir

- les LLM sont des machines probabilistes à prédire le *token* suivant, pas des oracles
- la fenêtre de contexte, la température et l'alignement façonnent ce que vous obtenez
- les hallucinations sont inhérentes : vérifier fait partie du travail

1. Le nouveau paradigme de développement
2. Les bases des LLM (pour le développeur)
3. Les bases du *prompting*
4. L'essentiel du *prompt engineering*
5. Pratiques du développement assisté par l'IA

- prompt = une requête en langage naturel adressée à un LLM
- c'est une forme de programmation
- le *prompt* est le nouveau point d'entrée :

  penser → taper → compiler devient décrire → générer → valider
- un LLM est une boite noire
- ... mais on a établi empiriquement des façons de communiquer efficaces

Quatre ingrédients :

Contexte	ce qui existe déjà
Intention	ce que vous voulez
Contraintes	langage, style, bibliothèques, cas limites
Format	la forme de la réponse

- même intention, formulation faible ou forte : des résultats sans commune mesure
- ambiguïté en entrée = ambiguïté en sortie
- être précis, c'est être clair, pas bavard

- ***zero-shot*** : décrire la tâche, sans exemple

prompt

Classe ce message de commit en `feat`, `fix` ou `docs` :
« corrige le débordement dans `parse_date` »

- ***zero-shot*** : décrire la tâche, sans exemple

prompt

> Classe ce message de commit en `feat`, `fix` ou `docs` :
> « corrige le débordement dans `parse_date` »

réponse du modèle

> Il s'agit d'un correctif, donc `fix` : le message décrit la correction d'un bug de débordement dans la fonction `parse_date`.

la réponse est juste, mais sa forme est libre : difficile à exploiter par un programme

- ***few-shot*** : $K > 0$ démonstrations de la tâche au moment de l'inférence, puis décrire la tâche
  - par exemple : K paires de traduction anglais/français
  - résultat clé [1] : $K > 0$ aide les modèles de langage génériques et les amènent (pour certaines tâches) au niveau des modèles spécialisés.

prompt

« ajoute l'export CSV » → `feat`
« documente l'option `--verbose` » → `docs`
« répare le test de connexion » → `fix`
« corrige le débordement de `parse_date` » → ?

1. T. B. Brown et al. « Language Models are Few-Shot Learners ». *NeurIPS*. 2020.

- ***few-shot*** : $K > 0$ démonstrations de la tâche au moment de l'inférence, puis décrire la tâche
  - par exemple : K paires de traduction anglais/français
  - résultat clé [1] : $K > 0$ aide les modèles de langage génériques et les amènent (pour certaines tâches) au niveau des modèles spécialisés.

prompt

« ajoute l'export CSV » → `feat`
« documente l'option `--verbose` » → `docs`
« répare le test de connexion » → `fix`
« corrige le débordement de `parse_date` » → ?

réponse du modèle

```
fix
```

$K = 3$ exemples fixent à la fois la tâche et le format de la réponse

1. T. B. Brown et al. « Language Models are Few-Shot Learners ». *NeurIPS*. 2020.

- où : le dépôt du cours
- installation : voir le `README.md` (modèle local avec Ollama)
- `exercices/cours1/` :
  - `k_shot_prompting.py`
  - `chain_of_thought.py`
  - `self_consistency_prompting.py`
  - `reflexion.py`



`https://github.com/otrack/public-aise`

Technique : *chain-of-thought*

- ***chain-of-thought*** : chaque exemple est accompagné d'un raisonnement pas à pas
  - décompose le problèmes et donne une fenêtre interprétable sur le raisonnement
  - robuste (aide toujours), utile en mathématiques, sens commun, raisonnement symbolique ; surtout précieux pour les problèmes complexes
  - ce comportement émerge avec les grands modèles (ayant plusieurs 10-100 de milliards de paramètres)
  - résultat clé [1] : permet aux modèles génériques d'atteindre un raisonnement arithmétique proche des modèles spécialisés

prompt

Q : 3 serveurs traitent chacun 40 requêtes/s. Il en arrive 150/s. Combien sont rejetées ?
R : $\textit{capacité} = 3 \times 40 = 120/s$; $150 - 120 = 30$. Réponse : 30.

Q : un service reçoit 120 requêtes/s ; chacune occupe un cœur pendant 50 ms. Combien de cœurs au minimum ?

1. J. Wei et al. « Chain-of-Thought Prompting Elicits Reasoning in Large Language Models ». *NeurIPS*. 2022.

Technique : *chain-of-thought*

- ***chain-of-thought*** : chaque exemple est accompagné d'un raisonnement pas à pas
  - décompose le problèmes et donne une fenêtre interprétable sur le raisonnement
  - robuste (aide toujours), utile en mathématiques, sens commun, raisonnement symbolique ; surtout précieux pour les problèmes complexes
  - ce comportement émerge avec les grands modèles (ayant plusieurs 10-100 de milliards de paramètres)
  - résultat clé [1] : permet aux modèles génériques d'atteindre un raisonnement arithmétique proche des modèles spécialisés

prompt

Q : 3 serveurs traitent chacun 40 requêtes/s. Il en arrive 150/s. Combien sont rejetées ?
R : *capacité* $= 3 \times 40 = 120/s$; $150 - 120 = 30$. Réponse : 30.

Q : un service reçoit 120 requêtes/s ; chacune occupe un cœur pendant 50 ms. Combien de cœurs au minimum ?

réponse du modèle

R : *un cœur traite 1000 / 50 = 20 requêtes/s ; 120 / 20 = 6.* Réponse : 6.

l'exemple montre *comment* raisonner ; le modèle reproduit les étapes intermédiaires

1. J. Wei et al. « Chain-of-Thought Prompting Elicits Reasoning in Large Language Models ». *NeurIPS*. 2022.

- ***self-consistency***[1] : tirer plusieurs réponses, avec une température non nulle pour explorer plusieurs chemins, et garder la majoritaire

prompt

> le *prompt chain-of-thought* précédent, envoyé 5 fois, avec une température de 0,7

1. X. Wang et al. « Self-Consistency Improves Chain of Thought Reasoning in Language Models ». *ICLR*. 2023.

Technique : *self-consistency*

- ***self-consistency***[1] : tirer plusieurs réponses, avec une température non nulle pour explorer plusieurs chemins, et garder la majoritaire

prompt

le *prompt chain-of-thought* précédent, envoyé 5 fois, avec une température de 0,7

réponse du modèle

1	1000 / 50 = 20 ; 120 / 20 = 6	**6**
2	120 × 0,05 = 6 cœurs occupés	**6**
3	120 / 50 = 2,4 ; arrondi à 3	3
4	20 requêtes/s par cœur ; 120 / 20	**6**
5	120 × 50 = 6000 ; 6000 / 1000	**6**

vote majoritaire : **6** (4 tirages sur 5)

les chemins de raisonnement diffèrent ; la majorité l'emporte sur le tirage isolé qui se trompe

1. X. Wang et al. « Self-Consistency Improves Chain of Thought Reasoning in Language Models ». *ICLR*. 2023.

Technique : reflexion

- **réflexion** [1] : après un échec, le modèle écrit une critique de *pourquoi* il a échoué, et cette critique est ajoutée au *prompt* de la tentative suivante
  - le signal d'échec vient de l'extérieur : des tests, un compilateur, un autre LLM
  - aucune mise à jour des poids : un « apprentissage par renforcement verbal »
  - résultat clé : 91 % de réussite (*pass@1*) sur le banc de test de code HumanEval, contre 80 % pour GPT-4 seul

prompt

```
def is_valid_password(p):
   return (len(p) >= 8
         and any(c.isdigit() for c in p))
```

Test échoué :
`is_valid_password("password1")` → `True`, attendu `False`.
Explique l'échec, puis corrige.

1. N. Shinn et al. « Reflexion : Language Agents with Verbal Reinforcement Learning ». *NeurIPS*. 2023.

- **réflexion** [1] : après un échec, le modèle écrit une critique de *pourquoi* il a échoué, et cette critique est ajoutée au *prompt* de la tentative suivante
  - le signal d'échec vient de l'extérieur : des tests, un compilateur, un autre LLM
  - aucune mise à jour des poids : un « apprentissage par renforcement verbal »
  - résultat clé : 91 % de réussite (*pass@1*) sur le banc de test de code HumanEval, contre 80 % pour GPT-4 seul

prompt

```
def is_valid_password(p):
   return (len(p) >= 8
         and any(c.isdigit() for c in p))
```

Test échoué :
`is_valid_password("password1")` → `True`, attendu `False`.
Explique l'échec, puis corrige.

réponse du modèle

Réflexion : *ma fonction ne vérifie pas la présence d'une majuscule, et « password1 » n'en contient pas.*

```
def is_valid_password(p):
   return (len(p) >= 8
         and any(c.isdigit() for c in p)
         and any(c.isupper() for c in p))
```

cette boucle sera revue avec les agents (cours 3)

1. N. Shinn et al. « Reflexion : Language Agents with Verbal Reinforcement Learning ». *NeurIPS*. 2023.

Autres motifs importants

Persona	« tu es un relecteur expérimenté » : donne au modèle un rôle, des exigences et un vocabulaire
Template	des squelettes de *prompts* paramétrés et réutilisables ; le motif à l'origine du *prompt programming* et des outils agentiques
Vérificateur cognitif	découper la question en sous-questions dont les réponses, ensemble, vérifient la question d'origine
Abstention contrôlée	répondre honnêtement « je ne sais pas », ou « il manque une information »
Longueur contrôlée	fixer la longueur maximale (ou minimale) de la réponse

- d'autres motifs existent [1] : raffinement de la question, approche alternative, jeu de rôle, liste de faits à vérifier, etc.
- les motifs se composent (*persona* + *template* pour relire du code)

1. J. White et al. *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. arXiv. 2023.

- un bon *prompt* : contexte, intention, contraintes, format
- une échelle de techniques :

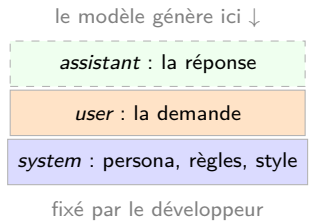
  zero-shot → *few-shot* → *chain-of-thought* → *self-consistency* → réflexion
- le *prompting* est un artisanat ; la *discipline* d'ingénierie (motifs, programmes, évaluation) vient ensuite

1. Le nouveau paradigme de développement
2. Les bases des LLM (pour le développeur)
3. Les bases du *prompting*
4. L'essentiel du *prompt engineering*
5. Pratiques du développement assisté par l'IA

- des *prompts* ponctuels, improvisés, ne se versionnent pas, ne se testent pas, ne se réutilisent pas → le code a déjà résolu tous ces problèmes !
- les *prompts* comme **artefacts logiciels à part entière** : écrits à la conception, exécutés avec des entrées utilisateur variables
- ***prompt programming*** : le « code » du programme est un *prompt* ; ses entrées et sorties sont des données
  - exemple : à partir de la transcription d'une réunion et d'un nom de participant, produire la liste de ses tâches et de leurs échéances
- analogie : les *prompts* sont l'API du modèle ; les programmes de *prompts* sont les applications construites dessus

Les rôles d'un *prompt* : *system*, *user*, *assistant*

- ***system*** : le premier message transmis au LLM, en général invisible pour l'utilisateur final
  - il fixe la *persona*, les règles sur la sortie, le style
- ***user*** : la demande ou l'instruction d'un humain
  - en pratique, tous nos exemples jusqu'ici
- ***assistant*** : la réponse du modèle
  - des *tokens* de contrôle invisibles (par exemple `<|im_start|>assistant`), appris lors de l'entraînement à la conversation, marquent le début de chaque rôle



ce que le développeur écrit (Python)

```
messages = [
 {"role": "system", "content":
  "Tu classes les messages de commit.
   Réponds uniquement par feat, fix
   ou docs."},
 {"role": "user", "content":
  "corrige le débordement
   dans parse_date"},
]
```

pour rappel : `[...]` est une **liste** Python, ici de deux **dictionnaires** `{...}` ; chacun associe à la clé `"role"` un rôle, et à la clé `"content"` un texte

ce que le développeur écrit (Python)

```
messages = [
 {"role": "system", "content":
  "Tu classes les messages de commit.
   Réponds uniquement par feat, fix
   ou docs."},
 {"role": "user", "content":
  "corrige le débordement
   dans parse_date"},
]
```

pour rappel : `[...]` est une **liste** Python, ici de deux **dictionnaires** `{...}` ; chacun associe à la clé `"role"` un rôle, et à la clé `"content"` un texte



ce que le modèle voit (format ChatML, celui de Qwen)

```
<|im_start|>system
Tu classes les messages de commit.
Réponds uniquement par feat, fix ou docs.
<|im_end|>
```

```
<|im_start|>user
corrige le débordement dans parse_date
<|im_end|>
```

```
<|im_start|>assistant
fix ← généré par le modèle
```

- **un *prompt* clair** : donnez-le à quelqu'un qui n'a presque pas de contexte ; s'il est perdu, le LLM le sera aussi
- **de la structure** : mettre en forme le *prompt* (sections, listes, balises)
- **la personnalité** (*role prompting*) : l'utiliser sans retenue, pour rendre les *system prompts* plus efficaces
- **être explicite** sur ce que vous voulez : langages, piles technologiques, bibliothèques, contraintes
- **décomposer** les tâches
- **lire les guides des fournisseurs de modèles**, par exemple celui de MistralAI
  https://docs.mistral.ai/inference/prompting

Bonnes pratiques : un prompt clair et structuré

mauvais prompt

Pourquoi ma fonction `map_users_by_id` ne marche pas ?

bon prompt

Contexte : ma fonction Python `map_users_by_id` doit transformer une liste d'utilisateurs en dictionnaire indexé par id.

Symptôme : elle plante avec `IndexError: list index out of range`.

Exemple : `[{"id": 1, "name": "Alice"}]` doit donner `{1: {"id": 1, "name": "Alice"}}`.

Code : `def map_users_by_id(users): ...`

Question : quel est le bug, et comment le corriger ?

→ démo : `demo_good_prompt.py`, le même bug, deux *prompts*

Bonnes pratiques : un prompt clair et structuré

mauvais prompt

Pourquoi ma fonction `map_users_by_id` ne marche pas ?

assistant

Pour diagnostiquer le problème, voici les étapes à vérifier :
1. la fonction est-elle bien définie ?
2. erreurs courantes : `KeyError`, `TypeError`, ...
3. vérifier les données d'entrée ...

bon prompt

Contexte : ma fonction Python `map_users_by_id` doit transformer une liste d'utilisateurs en dictionnaire indexé par id.

Symptôme : elle plante avec `IndexError: list index out of range`.

Exemple : `[{"id": 1, "name": "Alice"}]` doit donner `{1: {"id": 1, "name": "Alice"}}`.

Code : `def map_users_by_id(users): ...`

Question : quel est le bug, et comment le corriger ?

assistant

Le bug vient de `range(len(users) + 1)` : l'indice `len(users)` n'existe pas.
Il faut itérer sur `range(len(users))` :

```
for i in range(len(users)):
```

même modèle, même bug : sans contexte, le modèle devine ; avec contexte, symptôme et exemple, il trouve

Bonnes pratiques : la personnalité change la réponse

user (le même des deux côtés)

Écris une fonction Python qui renvoie `True` si un mot est un palindrome.

system

Tu es un assistant serviable qui adore programmer, au niveau d'un développeur senior, et tu es très détaillé et pédant dans tes réponses.

system

Tu es le BFF numérique d'une personne de la génération Z. Écris toujours comme si tu envoyais un message sur Snapchat à 2 h du matin.

→ démo : `demo_personna.py`, impact de la personnalité

Bonnes pratiques : la personnalité change la réponse

user (le même des deux côtés)

Écris une fonction Python qui renvoie `True` si un mot est un palindrome.

system

Tu es un assistant serviable qui adore programmer, au niveau d'un développeur senior, et tu es très détaillé et pédant dans tes réponses.

assistant

```
def is_palindrome(word: str) -> bool:
    """Vrai si word se lit pareil dans les
    deux sens, casse ignorée.
    >>> is_palindrome("Kayak")
    True"""
    w = word.casefold()  # gère aussi le « ß »
    return w == w[::-1]
# mémoire O(n) ; deux indices : O(1)
```

system

Tu es le BFF numérique d'une personne de la génération Z. Écris toujours comme si tu envoyais un message sur Snapchat à 2 h du matin.

assistant

```
# ok voilà frr c'est une ligne mdrr
def is_palindrome(mot):
    return mot == mot[::-1]  # no cap
# si c'est pareil à l'envers c'est validé
# dors maintenant
```

même demande, même modèle : seul le *system prompt* change, et avec lui le style du code et le niveau de détail

Bonnes pratiques : être explicite

system (le même des deux côtés)

Réponds uniquement par du code, sans explication.

mauvais prompt

Donne-moi la commande pour trouver les fichiers Python modifiés récemment.

bon prompt

Langage et pile : commande bash (Linux, GNU `find`).

Contraintes : fichiers `.py` du dossier `src/` et de ses sous-dossiers, modifiés il y a moins de 24 heures.

Format : un fichier par ligne.

→ démo : `demo_be_explicit.py`, la même demande, deux *prompts*

system (le même des deux côtés)

Réponds uniquement par du code, sans explication.

mauvais prompt

Donne-moi la commande pour trouver les fichiers Python modifiés récemment.

assistant

```
find /chemin/vers/le/dossier -type f
  -name "*.py" -mtime -7
  -exec ls -lt {} \;
  | awk '{print $5, $9}'
```

... puis une seconde variante avec `-newermt`

bon prompt

Langage et pile : commande bash (Linux, GNU `find`).

Contraintes : fichiers `.py` du dossier `src/` et de ses sous-dossiers, modifiés il y a moins de 24 heures.

Format : un fichier par ligne.

assistant

```
find src/ -type f -name "*.py" -mtime -1
```

ce qui n'est pas dit est inventé : dossier fictif, durée de 7 jours, format non demandé ; langage, pile, contraintes et format enlèvent ces choix

Pour aller plus loin : hygiène d'évaluation

- les *prompts* sont du code ⇒ ils méritent une suite de tests
- un **jeu de référence** (*golden set*) : des paires (entrée, résultat attendu)
- ***LLM-as-judge*** : un second modèle note les réponses selon une grille, quand l'« attendu » est flou
- **tests de non-régression** : rejouer le jeu de référence à chaque modification du *prompt*
- l'automatisation débute : l'optimisation automatique de *prompts* ne marche qu'une fois le jeu de référence en place

1. Le nouveau paradigme de développement
2. Les bases des LLM (pour le développeur)
3. Les bases du *prompting*
4. L'essentiel du *prompt engineering*
5. Pratiques du développement assisté par l'IA

- nous avons la théorie : ce que font les LLM, comment les solliciter, comment itérer
- comment utiliser l'IA au quotidien ?
  - le vibe coding
  - cinq cas d'usage
  - puis la relecture de code, le *pair programming*, et quand ne pas faire confiance au modèle

décrire → générer → relire → tester → affiner → recommencer

- affiner : dire *ce qui* ne va pas (pas seulement « c'est faux »), restreindre l'espace des solutions, une correction par *prompt*, ajouter le contexte manquant, des consignes négatives
- situations d'échec : *prompts* vagues, contexte manquant, API hallucinées, sur-génération
- déboguer un *prompt* : l'intention était-elle claire ? le contexte suffisant ? les contraintes explicites ? peut-on découper en *prompts* plus petits ?

Cas d'usage : génération de code

- code répétitif, squelettes, implémentations bien définies (routes, modèles de données, analyse des arguments d'une API)
- marche bien : tâche mécanique, interface claire ; peine sur : logique métier complexe, algorithmes nouveaux, invariants subtils
- conseil : générer par petites unités (une fonction, une classe) et composer

prompt

> Écris une route Flask `GET /users/<id>` qui renvoie l'utilisateur en JSON, ou une erreur 404 s'il n'existe pas.

Cas d'usage : génération de code

- code répétitif, squelettes, implémentations bien définies (routes, modèles de données, analyse des arguments d'une API)
- marche bien : tâche mécanique, interface claire ; peine sur : logique métier complexe, algorithmes nouveaux, invariants subtils
- conseil : générer par petites unités (une fonction, une classe) et composer

prompt

> Écris une route Flask `GET /users/<id>` qui renvoie l'utilisateur en JSON, ou une erreur 404 s'il n'existe pas.

réponse du modèle

```
@app.get("/users/<int:uid>")
def get_user(uid):
    user = db.get(uid)
    if user is None:
        abort(404)
    return jsonify(user)
```

une tâche mécanique, une interface claire : le terrain idéal

Cas d'usage : refactorisation

- restructurer sans changer le comportement (extraire une méthode, renommer)
- le modèle doit *voir* le code d'origine et les invariants à préserver
- conseil : toujours relancer les tests existants après une refactorisation assistée

prompt

```
def total(orders):
    s = 0
    for o in orders:
        if o.qty > 0 and o.price > 0:
            s += o.qty * o.price
    return s
```

Extrais le test de validité dans une fonction, sans changer le comportement.

Cas d'usage : refactorisation

- restructurer sans changer le comportement (extraire une méthode, renommer)
- le modèle doit *voir* le code d'origine et les invariants à préserver
- conseil : toujours relancer les tests existants après une refactorisation assistée

prompt

```
def total(orders):
    s = 0
    for o in orders:
        if o.qty > 0 and o.price > 0:
            s += o.qty * o.price
    return s
```

Extrais le test de validité dans une fonction, sans changer le comportement.

réponse du modèle

```
def is_valid(o):
    return o.qty > 0 and o.price > 0
def total(orders):
    return sum(o.qty * o.price
               for o in orders if is_valid(o))
```

même comportement ? ce sont les tests qui le disent, pas le modèle

Cas d'usage : débogage

- comprendre les messages d'erreur, localiser les causes probables, suggérer des corrections
- limites : corrections plausibles mais fausses, aucune vue sur l'état à l'exécution, difficile sur la concurrence, la mémoire
- conseil : traiter les suggestions comme des hypothèses à tester, pas comme des réponses à appliquer

prompt

```
def mean(xs):
    return sum(xs) / len(xs)
mean([]) lève ZeroDivisionError: division
by zero. Pourquoi ?
```

- comprendre les messages d'erreur, localiser les causes probables, suggérer des corrections
- limites : corrections plausibles mais fausses, aucune vue sur l'état à l'exécution, difficile sur la concurrence, la mémoire
- conseil : traiter les suggestions comme des hypothèses à tester, pas comme des réponses à appliquer

prompt

```
def mean(xs):
    return sum(xs) / len(xs)
mean([]) lève ZeroDivisionError: division
by zero. Pourquoi ?
```

réponse du modèle

```
# hypothèse : xs vide, len(xs) == 0
def mean(xs):
    if not xs:
        raise ValueError("liste vide")
    return sum(xs) / len(xs)
```

la cause est juste, mais la correction est un choix : que *doit* renvoyer `mean([])` ? c'est à vous de trancher

Cas d'usage : documentation

- *docstrings*, README, documentation d'API, explications en langage courant : l'un des usages les plus rentables, il supprime la corvée
- conseil : vérifier les faits — paramètres, types de retour, comportement dans les cas limites peuvent être hallucinés

prompt

```
def mean(xs):
    return sum(xs) / len(xs)
```

Écris la *docstring* de cette fonction.

Cas d'usage : documentation

- *docstrings*, README, documentation d'API, explications en langage courant : l'un des usages les plus rentables, il supprime la corvée
- conseil : vérifier les faits — paramètres, types de retour, comportement dans les cas limites peuvent être hallucinés

prompt

```
def mean(xs):
    return sum(xs) / len(xs)
```

Écris la *docstring* de cette fonction.

réponse du modèle

```
"""Moyenne arithmétique d'une liste.

Args:
    xs: une liste de nombres.
Returns:
    La moyenne des éléments de xs,
    ou 0.0 si xs est vide.
"""
```

la dernière ligne est hallucinée : sur une liste vide, le code lève `ZeroDivisionError`

Cas d'usage : écrire des tests

- tests unitaires ou par propriétés, cas limites, *fixtures* et *mocks* : mécanique et vérifiable, un usage idéal
- attention : le modèle teste ce que le code *fait*, pas ce qu'il *devrait* faire
- conseil : écrire la spécification des tests en langage naturel avant d'en générer le code

prompt

```
def is_palindrome(w):
    return w == w[::-1]
```

Écris des tests unitaires pour cette fonction.

Cas d'usage : écrire des tests

- tests unitaires ou par propriétés, cas limites, *fixtures* et *mocks* : mécanique et vérifiable, un usage idéal
- attention : le modèle teste ce que le code *fait*, pas ce qu'il *devrait* faire
- conseil : écrire la spécification des tests en langage naturel avant d'en générer le code

prompt

```
def is_palindrome(w):
    return w == w[::-1]
```

Écris des tests unitaires pour cette fonction.

réponse du modèle

```
def test_is_palindrome():
    assert is_palindrome("kayak")
    assert not is_palindrome("python")
    assert not is_palindrome("Kayak")
```

si la casse devait être ignorée, le dernier test fige le bug au lieu de le révéler

Relecture de code assistée

- l'IA en *premier passage* : bugs, style, failles, conventions
- le jugement humain par-dessus : l'intention du système, la *nécessité* du changement

***Pair programming* avec un LLM**

- l'un génère, l'autre relit
- faire confiance : tâches mécaniques, faciles à vérifier
- reprendre la main : sécurité, connaissances métier, logique difficile à vérifier

Des *templates* pour les tâches courantes

- un *template* = un *prompt* à trous, réutilisé pour une tâche qui revient : générer, refactoriser, déboguer, tester, expliquer
- le motif *Template* de la section précédente, au quotidien
- il impose les quatre ingrédients : contexte, intention, contraintes, format
- coller le code *et* l'erreur ; dire ce que vous attendiez, et ce que vous avez obtenu

un *template* de débogage

```
Contexte : <langage, bibliothèques, versions>
Code : <le code concerné>
Erreur : <le message complet, la trace>
Attendu : <ce que vous attendiez>
Obtenu : <ce que vous avez obtenu>
Question : quelle est la cause probable, et
quelle est la correction minimale ?
```

les trous, en rouge, sont remplis à chaque usage ; le reste ne change pas

- code critique pour la sécurité (authentification, cryptographie, contrôle d'accès)
- algorithmes nouveaux, correction qui repose sur une preuve
- chemins critiques pour la performance
- décisions d'architecture à grande échelle
- quand vous ne pouvez pas relire le résultat vous-même

L'IA génère. Vous validez.

- vous restez responsable de la correction, de la sécurité, de la nécessité et de la maintenabilité
- l'IA accélère le développement ; elle ne remplace pas la **responsabilité**

- répondez au quiz du cours 1, depuis votre téléphone ou votre ordinateur
- scannez le QR code, ou allez sur le lien ci-dessous



https://digistorm.app/p/3913705